Linnæus University
Sweden

Lab Assignment 4

C-programming and interrupts

Computer Technology 1

November 9, 2026



Author: Andres Rodriguez
Examiner: Thomas
Semester: HT2026
Discipline: Exchange Erasmus Student
Course code: 1dt301

GitHub Link

Listing 1: Our link:

```
https://github.com/Andresrgg/ComputerTechnology
```

Wiring explanation

```
In the first part of the exercise, we connect the 1st leg on the
   left of the Pico (GP0) to the anode of the LED, then connect the
   cathode through a resistor to the GND rail tied to the 3rd leg on
    the left (GND), while the first push button connects between the
    2nd leg on the left (GP1) and the GND rail to serve as the on
   switch, and the second push button connects between the 4th leg
   on the left (GP2) and the GND rail to serve as the off switch (
   pull up way).
Then in b) we reuse the wiring set up and then in part c), we add a
   second LED by connecting its anode to GP6 (9th leg on the left
   side of the Pico) and its cathode through another current-
   limiting resistor to the GND rail.
For the second exercise, we connect 4 leds with their anodes
   connected to GPIO pins GP1,GP2,GP3 and GP4, each having its
   cathode wired through a resistor to the common GND rail, while
   the increment button is connected between GP5 and GND, and the
   decrement button is connected between GP6 and GND, both using
   internal resistors and falling-edge GPIO interrupts so that
   pressing the GP5 button increases a 4-bit binary counter starting
    at 0 up to a limit of 15, and pressing the GP6 button decreases
   the counter down to a minimum of 0, displaying the binary value
```

```
   across the four LEDs.
Lastly, we use the same LEDs set up from task2 but we disconnect the
    buttons to connect a new pull up button in GP0 (first leg in the
    PICO) that uses an interrupt to restart the count.
```

Task 1a

C code

Listing 2: Input and output in C

```c
#include "pico/stdlib.h"

#define LED 0
#define B_ON 1
#define B_OFF 2

int main() {
   stdio_init_all();

   gpio_init(LED);
   gpio_set_dir(LED, GPIO_OUT);

   gpio_init(B_ON);
   gpio_set_dir(B_ON, GPIO_IN);
   gpio_pull_up(B_ON);

   gpio_init(B_OFF);
   gpio_set_dir(B_OFF, GPIO_IN);
   gpio_pull_up(B_OFF);
```

```
    while (true) {
        bool on = gpio_get(B_ON);
        bool off = gpio_get(B_OFF);

        if (!on && off) {
            gpio_put(LED, 1);
        } else if (!off && on) {
            gpio_put(LED, 0);
        }

        sleep_ms(50);
    }

    return 0;
}
```

Comments

In this task, pin 0 is configured as a digital output to drive an LED, while pins 1 and 2 are configured as inputs with internal pull-up resistors enabled for two push buttons (`B_ON` and `B_OFF`). Inside an infinite execution loop, the program continuously polls the logic state of both push buttons using `gpio_get()`. Conditional statements evaluate the sampled input states: when only `B_ON` is pressed (logic 0), the LED output is driven high via `gpio_put(LED, 1)`; when only `B_OFF` is pressed (logic 0), the LED is turned off by writing a low signal. A delay of 50 ms (`sleep_ms(50)`) is included at the end of each loop iteration to avoid bouncing.

Task 1b

C code

Listing 3: Direct hardware register control

```
#include "pico/stdlib.h"

#define LED 0
#define B_ON 1
#define B_OFF 2

#define SIO_BASE 0xD0000000u
#define SIO_GPIO_IN ((volatile uint32_t *)(SIO_BASE + 0x04))
#define SIO_GPIO_OUT_SET ((volatile uint32_t *)(SIO_BASE + 0x14))
#define SIO_GPIO_OUT_CLR ((volatile uint32_t *)(SIO_BASE + 0x18))

int main() {
   stdio_init_all();

   gpio_init(LED);
   gpio_set_dir(LED, GPIO_OUT);

   gpio_init(B_ON);
   gpio_set_dir(B_ON, GPIO_IN);
   gpio_pull_up(B_ON);

   gpio_init(B_OFF);
   gpio_set_dir(B_OFF, GPIO_IN);
   gpio_pull_up(B_OFF);
```

```
    while (true) {
        uint32_t inputs = *SIO_GPIO_IN;

        if (!(inputs & 0b00000010) && (inputs & 0b00000100)) {
            *SIO_GPIO_OUT_SET = 0b00000001;
        }
        else if (!(inputs & 0b00000100) && (inputs & 0b00000010)) {
            *SIO_GPIO_OUT_CLR = 0b00000001;
        }

        sleep_ms(50);
    }

    return 0;
}
```

Comments

In this task, we manipulate GPIO states by directly accessing the SIO hardware registers of the RP2040. Following pin directional configuration and pull-up initialization, the program continuously samples all input states simultaneously in a single read operation by dereferencing `SIO_GPIO_IN` at base address `0xD0000000`. Bitwise bitmask operations (`0b00000010` and `0b00000100`) isolate the input logic levels on GPIO 1 and GPIO 2. When the ON button is active, writing to `SIO_GPIO_OUT_SET` sets output pin 0 high; when the OFF button is active, writing to `SIO_GPIO_OUT_CLR` drives pin 0 low. Utilizing hardware set and clear registers prevents race conditions while maintaining a 50 ms polling delay to avoid bouncing.

Task 1c

C code

Listing 4: Multi-pin manipulation via SIO

```
#include "pico/stdlib.h"

#define LED1 0
#define B_ON 1
#define B_OFF 2

#define LED6 6

#define SIO_BASE 0xD0000000u
#define SIO_GPIO_IN ((volatile uint32_t *)(SIO_BASE + 0x04))
#define SIO_GPIO_OUT_SET ((volatile uint32_t *)(SIO_BASE + 0x14))
#define SIO_GPIO_OUT_CLR ((volatile uint32_t *)(SIO_BASE + 0x18))

int main() {
   stdio_init_all();

   gpio_init(LED1);
   gpio_set_dir(LED1, GPIO_OUT);

   gpio_init(B_ON);
   gpio_set_dir(B_ON, GPIO_IN);
   gpio_pull_up(B_ON);

   gpio_init(B_OFF);
```

```
    gpio_set_dir(B_OFF, GPIO_IN);
    gpio_pull_up(B_OFF);

    gpio_init(LED6);
    gpio_set_dir(LED6, GPIO_OUT);

    while (true) {
        uint32_t inputs = *SIO_GPIO_IN;

        if (!(inputs & 0b00000010) && (inputs & 0b00000100)) {
            *SIO_GPIO_OUT_SET = 0b01000001;
        }
        else if (!(inputs & 0b00000100) && (inputs & 0b00000010)) {
            *SIO_GPIO_OUT_CLR = 0b01000001;
        }

        sleep_ms(50);
    }

    return 0;
}
```

Comments

In this task, direct SIO register manipulation is extended to demonstrate simultaneous control across multiple output lines using multi-bit bitmasks. Pins 0 (`LED1`) and 6 (`LED6`) are configured as outputs, while pins 1 (`B_ON`) and 2 (`B_OFF`) act as pull-up inputs. Inside the execution loop, input logic levels are read from memory-mapped register

SIO_GPIO_IN. When the ON button state is detected, writing the bitmask 0b01000001 to SIO_GPIO_OUT_SET atomically forces target output pins 0 and 6 high in a single bus transaction. Conversely, when the OFF button state is detected, writing the bitmask to SIO_GPIO_OUT_CLR clears those output bits simultaneously. Each loop iteration concludes with a 50 ms sleep to avoid button bouncing.

Task 2

C code

Listing 5: Binary counter using interrupts

```c
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#define LED1 1
#define LED2 2
#define LED3 3
#define LED4 4

#define B_INC 5
#define B_DEC 6

volatile int counter = 0;

void update_leds(int val) {
   int bit1 = (val >> 0) & 1;
   int bit2 = (val >> 1) & 1;
```

```
    int bit3 = (val >> 2) & 1;
    int bit4 = (val >> 3) & 1;

    gpio_put(LED1, bit1);
    gpio_put(LED2, bit2);
    gpio_put(LED3, bit3);
    gpio_put(LED4, bit4);
}

void gpio_callback(uint gpio, uint32_t events) {
    if (gpio == B_INC) {
        if (counter < 15) {
            counter++;
        }
    } else if (gpio == B_DEC) {
        if (counter > 0) {
            counter--;
        }
    }
    update_leds(counter);
}

int main() {
    stdio_init_all();
    for (int i = LED1; i <= LED4; i++) {
        gpio_init(i);
        gpio_set_dir(i, GPIO_OUT);
    }
```

```
    gpio_init(B_INC);
    gpio_set_dir(B_INC, GPIO_IN);
    gpio_pull_up(B_INC);

    gpio_init(B_DEC);
    gpio_set_dir(B_DEC, GPIO_IN);
    gpio_pull_up(B_DEC);

    update_leds(counter);

    gpio_set_irq_enabled_with_callback(B_INC, GPIO_IRQ_EDGE_FALL,
        true, &gpio_callback);
    gpio_set_irq_enabled_with_callback(B_DEC, GPIO_IRQ_EDGE_FALL,
        true, &gpio_callback);

    while (1) {
        tight_loop_contents();
    }
}
```

Comments

In this task, we construct a 4-bit UP/DOWN binary counter utilizing hardware interrupts instead of active polling. Four LEDs attached to GPIO pins 1 through 4 display the current binary representation of a global `counter` variable. Push buttons connected to GPIO 5 (`B_INC`) and GPIO 6 (`B_DEC`) are configured with pull-up resistors and set to trigger falling-edge interrupts (`GPIO_IRQ_EDGE_FALL`). When a button press interrupts exe-

cution, `gpio_callback()` determines which pin triggered the signal and safely increments or decrements `counter` within a bounded range (0 to 15). The helper function `update_leds()` uses bit-shifting and bitwise AND operations (`(val » n) & 1`) to isolate individual bits and write them to the corresponding GPIO outputs, while `main()` yields CPU cycles in `tight_loop_contents()`.

Task 3

C code

Listing 6: Binary counter with timer alarm and reset button

```
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#define LED1 1
#define LED2 2
#define LED3 3
#define LED4 4

#define B_INC 0

volatile int counter = 0;

void update_leds(int val) {
   int bit1 = (val >> 0) & 1;
   int bit2 = (val >> 1) & 1;
   int bit3 = (val >> 2) & 1;
```

```
    int bit4 = (val >> 3) & 1;

    gpio_put(LED1, bit1);
    gpio_put(LED2, bit2);
    gpio_put(LED3, bit3);
    gpio_put(LED4, bit4);
}

void gpio_callback(uint gpio, uint32_t events) {
    if (gpio == B_INC ) {
        counter = 0;
        update_leds(counter);
    }
}

int64_t timer_callback(alarm_id_t id, void *unused) {
    counter++;
    if (counter > 15) {
        counter = 15;
    }
    update_leds(counter);
    return 1000000; // 1 second in microseconds
}

int main() {
    stdio_init_all();
    for (int i = LED1; i <= LED4; i++) {
        gpio_init(i);
```

```
        gpio_set_dir(i, GPIO_OUT);
    }

    gpio_init(B_INC );
    gpio_set_dir(B_INC , GPIO_IN);
    gpio_pull_up(B_INC );

    update_leds(counter);

    gpio_set_irq_enabled_with_callback(B_INC , GPIO_IRQ_EDGE_FALL,
        true, &gpio_callback);
    add_alarm_in_ms(1000, timer_callback, NULL, false);

    while (1) {
        tight_loop_contents();
    }
}
```

Comments

In this task, we integrate hardware timer alarms with a GPIO edge interrupt to implement an automated 4-bit binary counter equipped with a manual reset function. A timer alarm initialized via `add_alarm_in_ms()` executes `timer_callback()` every 1000 ms (1 second), incrementing the global `counter` up to 15 and updating the LED indicators on GPIO pins 1 to 4. Simultaneously, a hardware interrupt is registered on GPIO 0 (`B_INC`) listening for falling-edge events (`GPIO_IRQ_EDGE_FALL`). When triggered by a button press, `gpio_callback()` resets `counter` to 0 and immediately refreshes the output LEDs.